

```

from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from pyspark.sql.types import DoubleType
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
from pyspark.ml.evaluation import BinaryClassificationEvaluator

print("1. Εκκίνηση αυτόνομης SparkSession για Logistic Regression...")
spark = SparkSession.builder \
    .appName("LR_SMOTE_GridSearch") \
    .master("local[*]") \
    .config("spark.driver.memory", "8g") \
    .getOrCreate()

print("2. Φόρτωση Δεδομένων (Απευθείας από το SMOTE Gold dataset)...")
train_gold = spark.read.parquet("../data/train_gold_with_cluster.parquet")
test_gold = spark.read.parquet("../data/test_gold_with_cluster.parquet")

# Μετατροπές τύπων για απόλυτη συμβατότητα
train_gold = train_gold.withColumn("stroke",
train_gold["stroke"].cast(DoubleType()))
test_gold = test_gold.withColumn("stroke",
test_gold["stroke"].cast(DoubleType()))
train_gold = train_gold.withColumn("cluster",
F.col("cluster").cast(DoubleType()))
test_gold = test_gold.withColumn("cluster", F.col("cluster").cast(DoubleType()))

print("3. Feature Augmentation (Ενσωμάτωση K-Means Cluster)...")
assembler = VectorAssembler(inputCols=["features", "cluster"],
outputCol="augmented_features")

train_augmented =
assembler.transform(train_gold).drop("features").withColumnRenamed("augmented_features",
"features")
test_augmented =
assembler.transform(test_gold).drop("features").withColumnRenamed("augmented_features",
"features")

# Caching του πλήρους SMOTE dataset
train_augmented.cache()
test_augmented.cache()

print("4. Ορισμός Logistic Regression και Πλέγματος Παραμέτρων (Grid Search)...")
lr_base = LogisticRegression(featuresCol="features", labelCol="stroke")

# Δημιουργία πλέγματος: Δοκιμάζουμε L1, L2 και ElasticNet σε διάφορες εντάσεις
ποινής

```

```

paramGrid = (ParamGridBuilder()
              .addGrid(lr_base.regParam, [0.01, 0.1, 1.0])           # Ένταση
ποινής
              .addGrid(lr_base.elasticNetParam, [0.0, 0.5, 1.0])     # 0.0=L2
(Ridge), 1.0=L1 (Lasso), 0.5=ElasticNet
              .addGrid(lr_base.maxIter, [100])                       # Μέγιστες
επαναλήψεις
              .build())

# Χρήση PR-AUC (areaUnderPR) για τη βέλτιστη ισορροπία ακρίβειας και ανάκλησης
evaluator = BinaryClassificationEvaluator(
    labelCol="stroke",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderPR"
)

cv = CrossValidator(estimator=lr_base,
                    estimatorParamMaps=paramGrid,
                    evaluator=evaluator,
                    numFolds=3,
                    seed=22390225)

print("5. Εκτέλεση Cross-Validation στο Πλήρες SMOTE Dataset...")
cv_model = cv.fit(train_augmented)
best_lr = cv_model.bestModel

print("\n" + "="*60)
print("[ΕΥΡΕΣΗ ΠΑΡΑΜΕΤΡΩΝ LOGISTIC REGRESSION ΟΛΟΚΛΗΡΩΘΗΚΕ]")
print(f" -> Βέλτιστο regParam (Ένταση Ποινής):
{best_lr._java_obj.getRegParam()}")
print(f" -> Βέλτιστο elasticNetParam (Τύπος Ποινής):
{best_lr._java_obj.getElasticNetParam()}")
print("="*60 + "\n")

print("6. Παραγωγή προβλέψεων στο άγνωστο Test Set...")
lr_preds = best_lr.transform(test_augmented)

print("7. Αποθήκευση των τελικών προβλέψεων...")
output_path = "../data/lr_predictions.parquet"
lr_preds.select("stroke", "prediction",
"probability").write.mode("overwrite").parquet(output_path)

print(f"=====")
print(f"[ΕΠΙΤΥΧΙΑ] Η βέλτιστη Λογιστική Παλινδρόμηση αποθηκεύτηκε!")
print(f"Αρχείο: {output_path}")
print(f"=====")

spark.stop()

```

1. Εκκίνηση αυτόνομης SparkSession για Logistic Regression...

```

WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/06/09 22:21:14 WARN Utils: Your hostname, cachyos-x8664, resolves to a
loopback address: 127.0.1.1; using 192.168.1.5 instead (on interface enp4s0)
26/06/09 22:21:14 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
26/06/09 22:21:15 WARN NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable

2. Φόρτωση Δεδομένων (Απευθείας από το SMOTE Gold dataset)...
3. Feature Augmentation (Ενσωμάτωση K-Means Cluster)...
4. Ορισμός Logistic Regression και Πλέγματος Παραμέτρων (Grid Search)...
5. Εκτέλεση Cross-Validation στο Πλήρες SMOTE Dataset...

```

```

=====
[EΥΡΕΣΗ ΠΑΡΑΜΕΤΡΩΝ LOGISTIC REGRESSION ΟΛΟΚΛΗΡΩΘΗΚΕ]
-> Βέλτιστο regParam (Ένταση Ποινής): 0.01
-> Βέλτιστο elasticNetParam (Τύπος Ποινής): 0.5
=====

```

```

6. Παραγωγή προβλέψεων στο άγνωστο Test Set...
7. Αποθήκευση των τελικών προβλέψεων...

```

```

=====
[ΕΠΙΤΥΧΙΑ] Η βέλτιστη Λογιστική Παλινδρόμηση αποθηκεύτηκε!
Αρχείο: ../../data/lr_predictions.parquet
=====

```